

315821001_MMIP

以下皆為讓 AI 做出初步架構，再依照個人目標做調整:

Quiz1

Quiz1 自行選擇一張彩色影像，將 RGB 彩色影像轉換為灰階影像。使用 NumPy 自行實作灰階轉換演算法，並與 OpenCV 提供的方法進行比較。●建議重複執行 N 次並取平均值，比較 (1) 執行速度、(2) 轉換結果、(3) 兩種方法之間的差異

Created a file, ran a command >

Runs perfectly — with the correct BT.601 weights, the NumPy result matches OpenCV exactly (0 pixel difference).

Ran a command, shared files >

`Codes/Quiz1.py` 完成，同樣符合 `run_quiz1(image_path, output_dir, N)` 的介面。

實作重點：

1. `numpy_rgb_to_gray()`：用 NumPy 自行實作，採用 OpenCV 內部使用的同一組加權係數 (ITU-R BT.601)： $\text{Gray} = 0.299R + 0.587G + 0.114B$ ，並做四捨五入與範圍裁切。
2. `opencv_rgb_to_gray()`：直接呼叫 `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` 對照。
3. 重複執行 N 次取平均執行時間比較速度。
4. 繪圖：輸出一張含 4 張子圖的比較圖——原始彩色影像、NumPy 灰階結果、OpenCV 灰階結

Quiz2

python中main.py /Data/Quiz3.jpg, /Codes/Quiz2.py

Quiz2

選擇一張影像，使用 Histogram Equalization 提升影像的明暗對比與動態範圍。●繪製影像在 Histogram Equalization 處理前後的灰階 Histogram，觀察並比較亮度分布的差異。需要:●使用 NumPy 自行實作演算法，並與 OpenCV 提供的方法進行比較。●建議重複執行 N 次並取平均值，比較 (1) 執行速度、(2) 影像增強效果、(3) 兩種方法之間的差異

main.py:

Show more

Ran 2 commands, created a file >

Code runs correctly. Let me copy it to the outputs directory matching your project structure.

Ran a command, shared files >

我已經寫好 `Codes/Quiz2.py`，符合你 `main.py` 的呼叫方式 `run_quiz2(image_path, output_dir, N)`。

實作重點：

Quiz3

新增Quiz3

選擇一張斜拍影像，利用 **Perspective Transformation** 將影像校正為正視影像。後續還要可以法應用於多張不同拍攝條件的影像，讓同一套流程可以針對多張影像自動完成梯形校正
可參考

Gaussian Blur → Canny → 輪廓偵測 → 梯形校正

Designing batch handling for single or multiple images. >

了解，Quiz3 是「斜拍影像 → 透視校正 (trapezoid correction)」，並且要能同時處理單張或多張不同拍攝條件的影像，用同一套自動化流程。我幫你寫成 `Codes/Quiz3.py`，流程依照你給的參考：

Gaussian Blur → Canny → 輪廓偵測 → 找出四邊形 (梯形) → Perspective Transform 校正

為了讓同一套函式能同時支援單張影像或整個資料夾的批次處理，我讓 `run_quiz3` 自動判斷 `image_path` 是檔案還是資料夾。

Quiz4

python中main.py /Data/Q4-1.jpg, /Codes/Quiz1.py
Quiz4:選擇兩張具有重疊區域的影像，使用 SIFT 等特徵偵測與匹配方法完成影像拼接

Building a Python image stitching script using SIFT matching.

我幫你寫一個使用 SIFT 特徵偵測與匹配來完成影像拼接 (Image Stitching) 的 `main.py`。我假設你有兩張重疊影像，預設路徑用 `/Data/Q4-1.jpg` 和 `/Data/Q4-2.jpg` (如果檔名不同，可以用參數指定)。

Created a file, shared files >

執行方式：

```
bash  
python main.py --img1 /Data/Q4-1.jpg --img2 /Data/Q4-2.jpg --out
```

Quiz4我要嘗試在影像拼接前加入適當的前處理方法，提升特徵偵測、匹配與拼接的穩定性。
可以再 `run_quiz4` 上加上 `preprocessing` 的參數嗎

Deciding default settings for the new preprocessing option. >

好的，我幫你在 `run_quiz4` 加上 `preprocessing` 參數，讓你可以開關/組合不同的前處理方法 (去雜訊、CLAHE 對比度增強、銳化)，用來提升 SIFT 特徵偵測與匹配的穩定性。

設計重點：前處理只用於「特徵偵測階段」，最終 `warp`/融合仍使用原始影像，避免前處理造成的色偏或過度銳化影響拼接結果的視覺品質；且前處理不改變座標系統，偵測到的關鍵點座標可直接套用回原圖。